Proof Search, Checking, Resources, and Semantics
MM0, HOL Light, and Computational Kernels

MeTTapedia research notes

15 October 2026

Abstract

A proof framework can host another faithfully while changing the size of its certificates, the work of its checker, and the representation of its models. We separate these obligations and describe their composition. The proposed organization is a resource-sensitive graph of configured proof systems, whose simulations induce preorders after choosing a resource class. MM0 and HOL Light supply explicit-certificate reference systems; Lean supplies an important computational comparison. Binding, theory composition, operational observations, and resource-bounded reasoning are connected without identifying proof adequacy with semantic completeness. These notes state a theory to develop, distinguish existing Lean results from remaining joins, and give positive examples and boundaries.

1 Four costs and three kinds of correctness

Consider a theorem with a short Lean proof using a substantial definitional computation. The stored proof can be small because the checker performs that computation. Expanding it into explicit elementary justifications can move work from the checker into the certificate and its producer. This is a possible trade-off, not a universal compression theorem. Explicit equality proofs can also be short, and a different encoding may share the work. Deduction modulo provides precise examples of proof-length speed-up through computation [3]; results about arithmetic proof speed-up depend on their specified theories and proof measures [2].

There are four separate accounts:

1. *Discovery:* search for a certificate, including tactic work.
2. *Storage:* physical bytes of statements, certificates, signatures, dependency tables, and shared libraries.
3. *Translation:* execution of an actual certificate compiler.
4. *Checking:* execution of the receiving checker.

Undecidable search is compatible with decidable checking and short proofs of some instances. This is already true in first-order logic; higher-order syntax is not the explanation for undecidability by itself. Nor does any of these facts give an algorithm for finding all short proofs quickly.

Three correctness statements must likewise remain distinct: *certificate preservation*, *derivability reflection*, and *semantic soundness or completeness*. Their different quantifiers are the central issue of the next two sections.

2 Primitive discipline and executed resource class

A kernel profile has two distinct coordinates. Its *primitive discipline* records the operations admitted at the selected observation: rule instantiation, syntax comparison, scope and dependency checks, substitution, explicit equality inference, or implicit conversion. Its *resource class* records proved bounds on the implemented checker under a specified physical encoding. The observations may be rule-level calls or smaller machine instructions, but the level must be stated.

The source-derived HOL Light inference interface supplies a concrete first case. `HOLLight.CheckingDiscipline` records a rule-level operation view of its ten existing inference requests. The required basis is minimal for this fixed view: a basis covers all ten requests exactly when it contains the six required operations. A basis excluding term substitution or explicit BETA admission does not cover the interface. BETA admission is an explicit rule request, not an implicit normalization service; none of the ten requests exposes implicit normalization. This classification excludes the separate axiom and declaration-history interfaces, and proves no instruction lower bound or running-time theorem.

Minimal inventories and generating bases. The word "minimal" has several distinct meanings. An inventory is least for an observer when every covering inventory contains it. A generating basis is inclusion-minimal when removing a rule destroys its ability to derive the required rules. A least generating basis would additionally have to be contained in every sufficient basis. Logical weakness, measured by assumptions or model classes, and resource efficiency are further orders; none follows from counting primitive names.

Rule generation is defined in `Logic.InferenceBasis` using derivations from each rule's premises. Cut substitutes derivations for those premises. Generation is equivalent to preservation of consequences under all open hypotheses, and composes. A finite candidate set has a minimal generating subset; irredundancy characterizes minimality.

Two concrete bases are given in `InferenceBasisControls`. One uses zero and successor; the other uses one, predecessor and addition by two. Both are minimal within their common union and have the same consequences, but there is no least sufficient subset under literal rule inclusion. These are constructed controls, not minimality theorems for MM0 or HOL Light.

`InferenceBasisBoundary` proves that primitive elimination is not computable in general. One rule asserts zero; the other requires checked finite evidence that a supplied program halts. Both local witness checks are decidable. The assertion rule can be removed exactly when the program halts. The positive control is a terminating zero program; the negative control searches forever for zero from a constant-one program. Finite existence of a minimal basis therefore supplies no general algorithm for finding it.

`Logic.InferenceBasisCompiler` provides explicit proof templates with indexed premise holes. Filling the templates with compiled children gives accepted target certificates with the original conclusions. Its interface states functional correctness; computability and executed clocks require realizations of its template function. The concrete control compiles zero/successor proofs into one/predecessor/add-two proofs and exactly doubles the tree-node count. Repeated holes charge repeated child occurrences; this construction makes no implicit graph-sharing claim. Node counts, serialized bytes and machine work remain different quantities.

Actual binding guards. The MM0 and Metamath profiles observe their original witness interfaces. MM0 includes simultaneous substitution, sorts, dependency preservation, explicit conversion witnesses and fresh dummy checks. Metamath includes substitution, typecodes and distinct-variable constraints. In both actual checkers, concrete substitutions pass the typing checks but violate the independence guard. Distinct images supply positive controls;

coincident images and missing declared independence supply negative ones. Thus these guards do substantive work. Their necessity under removal does not prove that the corresponding whole kernels are semantically irredundant or physically linear-time.

`KernelCheckingClasses` independently defines linear checking from the original executed clock on complete encoded packets. Linear checking implies polynomial checking. A linear exact translation followed by a linear checker also supplies linear checking, including refused inputs. The derived factor charges the source machine's syntactic push bound, the written target packet, and the physical handoff. With only a polynomial translation bound, the corresponding conclusion is polynomial checking. Mutual exact polynomial simulation preserves that checking class without identifying exact costs, primitive disciplines, axioms or proof-search strategies.

Exact authority alone does not give polynomial equivalence. The existing unary and binary evenness presentations have identical Boolean acceptance and exact authority for the same predicate, but translation to the unary certificate format cannot have a polynomial output-size bound. The new `KernelComparisonBoundary` exports this as a counterexample to collapsing all exact checkers with polynomial declared time accounts into one size class. These accounts are not silently promoted to original-machine clocks. A universal least kernel therefore needs a stated class and comparison relation; it cannot follow merely from effective checking or expressive reach.

Searching partial checkers. The input to `EffectiveKernelSearch` is a finite code for a partial checking program. Acceptance means that evaluation of the exact claim/certificate pair returns the designated value 1. A finite bounded-evaluation witness makes checking a submitted candidate primitive recursive. Fair search enumerates both candidate certificates and their bounds, so a divergent candidate cannot trap the search. Its successful domain is exactly the set of claims accepted by that program. Soundness transfers found evidence to semantic truth; semantic completeness is a separate hypothesis. A non-recursively-enumerable semantic validity predicate has no effective checker exact for all its claims. The supplied evaluation bound is not an original-machine time bound. The concrete `NonEffectiveKernelBoundary` case supplies effective finite evidence for halting but excludes a sound and complete effective finite evidence language for all nonhalting claims, even with a partial checker. This does not exclude sound proofs of particular nonhalting claims or regular infinitary certificates within an effectively checked scope.

`FiniteCertificateSearch` enumerates all binary words of length at most n, with exactly $2^{n+1}-1$ candidate entries. Exhaustion excludes this finite collection. A separately proved sufficient proof-length bound turns exhaustion into underivability. A size simulation transports a target search radius, but this does not make exhaustive search polynomial or synthesize an efficient proof compressor.

`HOLLight.EffectiveKernelSearchBridge` connects successful partial-checker search to the existing executable evidence extractor and shared HOL graph compiler. The same fixed finite hosting theory accepts exactly the quoted checking-code evaluations. Its reflection theorem covers arbitrary accepted HOL certificates, not only this compiler's output. Source derivability and model truth still require independent code-to-source authority. The executable controls include a successful 761-receipt graph and refusal of a changed root, wrong source result and insufficient evidence. No polynomial expansion in the original source proof is asserted.

Heuristics and standard truth. `HeuristicEvidence` screens arbitrary submitted certificates with the original checker. Soundness requires no correctness or completeness of the producer. Screening composes over batches while retaining each accepted occurrence. Empty

results exclude only those submissions. For every fixed sound effective checker of nonhalting, a true nonhalting claim has no accepted certificate; even an abstract oracle proposing arbitrary finite certificates cannot close that gap without changing the checker. The quantifier is "for each checker, some missing truth," not "one truth unprovable in every sound theory."

`HOLLight.KernelAtlas` joins screened HOL graphs, the actual HOL-to-MM0 certificate compiler, its established structural size bound, and truth in the constructed full-domain model. The individual carrier is explicitly equivalent to the ordinary natural numbers. No new executed translation clock is supplied. Eta provides a concrete model-valid sentence without a certificate in the *axiom-free* configuration; adjoining ETA changes that configuration. The example distinguishes exact hosting from semantic completeness without questioning sound proofs about standard models.

Finite budgets and explicit conversion evidence. `FuelledConversionEvidence` observes the actual finite-code evaluator with three outcomes: accepted, completed refusal, and exhausted. Completed results persist at larger bounds. Every result of the original partial program eventually appears at a finite bound, including refusals. This is completeness for that program's evaluation relation; it does not establish termination or semantic completeness. Evaluator depth is not an executed machine clock or a Lean heartbeat count.

The modeled Mathdata normalizer has its own monotonicity and completion theorems in `FuelledConversionProfile`. Successful comparisons produce articles accepted by the original explicit conversion checker. Its fixed-pass successful comparison is transitive. By contrast, `FuelledConversionWitness` supplies a selected definition-unfolding search whose one-step recognizer accepts $a \sim b$ and $b \sim c$ but cannot find $a \sim c$ until its budget is enlarged. The checked paths themselves compose with additive length budgets and reverse without losing their original witnesses. A completed failed search excludes that selected route; it is not a proof of inequivalence under every conversion rule.

A concrete raw environment with a self-referential definition exhausts every unfolding bound. This is a boundary for arbitrary supplied environments, not an admitted acyclic theory history or a counterexample to termination of a typed fragment. The source converter's counters also differ from modeled pass fuel. The inspected Megalodon source uses a lazy converter and resource checks; Proofgold's pass loop has a distinct zero-budget convention, for which `strictPass_offset` proves the exact offset for the modeled pass operation [25, 26].

Digest identifiers and certificates. `CertificateDigestBoundary` proves that exact replay based solely on a digest exists set-theoretically precisely when original acceptance is constant on each digest fibre, separately for every claim. An accepted and rejected certificate with the same digest prevents such replay. The converse uses classical representative selection and gives no efficient algorithm. A certificate for computation, an authenticated cache and a cryptographic argument consequently need their own correctness and trust contracts.

3 Configured checkers and faithful hosting

Definition 1 (Configured effective checker)**.** *A configuration consists of an effectively represented signature and theory Θ, effectively represented claims J_Θ and finite certificates P_Θ, and an implemented Boolean checking algorithm $\mathsf{check}_\Theta(j, p)$. Unless explicitly relaxed, the algorithm terminates on* all *inputs, including malformed certificates. Define*

$$\mathsf{Prov}_\Theta(j) \quad \Longleftrightarrow \quad \exists p \in P_\Theta,\ \mathsf{check}_\Theta(j, p) = \mathsf{true}.$$

The implementation's termination and correspondence to a mathematical checker are additional verification obligations.

For a partial implementation, use the relation "there is a finite accepting run" instead. Enumeration then dovetails certificates and execution bounds; an unfinished run is not a rejection. We must use this variant whenever total termination of the chosen implementation is not established. This distinction is relevant to computational conversion, not just to search.

An MM0 theory is a configuration of a framework. A HOL theory configures a fixed HOL kernel. OpenTheory supplies an exchange and theory-composition layer for the HOL family. Lean also includes a computational conversion discipline. Comparisons of these systems therefore quantify over declared configurations; they do not compare unqualified bare kernels with arbitrary axioms silently supplied to one side. For a uniform framework translation, also give a computable map Φ from admitted source configurations to admitted target configurations. Its initialization time and emitted declaration size are charged separately or included in the variable input. A collection of fixed-theory proof bounds is not automatically a uniform bound for compiling arbitrary `.module` declarations.

Definition 2 (Hosting data and obligations)**.** *For configured systems S, T, give a claim translation $\tau : J_S \to J_T$ and certificate compiler $E : J_S \times P_S \to P_T$. The following are independent obligations:*

$$\begin{aligned} \mathsf{check}_S(j,p) = \mathsf{true} &\implies \mathsf{check}_T(\tau j, E(j,p)) = \mathsf{true}, && \textit{certificate preservation;} \\ \mathsf{Prov}_T(\tau j) &\implies \mathsf{Prov}_S(j), && \textit{global derivability reflection.} \end{aligned}$$

Faithful hosting satisfies both, so $\mathsf{Prov}_S(j) \iff \mathsf{Prov}_T(\tau j)$.

The first statement concerns outputs of E. The second concerns *every* target proof of an encoded source claim, including proofs that E could never produce. For example, interpreting an authored calculus's patterns as data and its predicate holds as source derivability proves reflection by a model argument. It does not supply an efficient algorithm extracting a source proof from an arbitrary HOL proof.

A further, useful certificate condition is

$$\mathsf{check}_S(j,p) = \mathsf{true} \quad \iff \quad \mathsf{check}_T(\tau j, E(j,p)) = \mathsf{true}.$$

This catches accepted translations of rejected source inputs. It still does not replace global reflection: a target might prove an encoded claim by some other certificate. Conversely, faithful hosting alone does not force E to preserve rejection of every malformed input. We require both conditions where a public translator promises exact rejection.

4 Resource grades and composition

Fix physical encodings and let $n = |j| + |p|$ count input bytes or bits consistently. Resource assertions include the configuration's size when it is variable input; fixed-theory bounds may put it in constants. Do not charge a variable module library as a free oracle.

A resource grade (a, f, g) bounds the translated claim size, certificate size, and *executed* translation work:

$$|\tau j| \le a(n), \qquad |E(j,p)| \le f(n), \qquad \mathsf{steps}_E(j,p) \le g(n).$$

The third inequality must follow from a machine executing τ and E on the same physical representations. A separately declared function named "translation time" establishes no such connection. All bounds state whether their input domain includes refusal paths.

Proposition 3 (Composition of resource bounds)**.** *Let $E_1 : S \to T$ and $E_2 : T \to U$ be simulations with nondecreasing grades (a_1, f_1, g_1) and (a_2, f_2, g_2). Suppose their physical handoff has a proved cost bound h. Put $m(n) = a_1(n) + f_1(n)$. The composite has bounds*

$$a_2(m(n)), \qquad f_2(m(n)), \qquad g_1(n) + h(m(n)) + g_2(m(n)).$$

Certificate preservation and global reflection each compose.

Proof. The second machine receives at most $m(n)$ units of data. Monotonicity gives its two size bounds and running bound; the first run and handoff are also charged. Acceptance preservation composes by implication. Reflection applies first to $T \to U$, then to $S \to T$. □

Thus polynomial size and executed-time simulations compose when the machine handoff is polynomial. Identity copying is an actual linear-cost implementation, not a zero-cost axiom. Suitable grades form a preorder after quantifying over available translations. Quotienting by simulations in both directions gives equivalence classes *at that resource grade*. Multiple incomparable grades or trust policies give a graph rather than a single numerical ranking. The richer proof-relevant category retains the chosen translations; its preorder forgets that evidence.

For the implemented stack-machine model, polynomial output size can be derived from executed polynomial time. Let c_M be the maximum number of syntactic pushes along a command branch, over the machine's finite labels. For a run on input x taking at most t transitions,

$$|\mathsf{output}| \le \mathsf{peakCells} \le |x| + c_M t.$$

The refined peak follows primitive mutations inside commands and all stacks, including scratch cells released before the next transition. A macrostep may contain several pushes; the checked three-push control rules out silently setting $c_M = 1$. A second control pushes and pops within one transition: its transition-state peak is zero, while its refined peak is one. Finite internal alphabets preclude hiding an arbitrary term in one cell. Native byte allocation and representation overhead require further refinements.

The original sequential-composition machine also has an exact storage law. Both component executions end with their scratch arenas empty, and each transfer step moves a symbol from one stack to another. Consequently its transition-state peak is

$$S_{\text{sequence}} = \max(S_{\text{first}}, S_{\text{second}}).$$

The handoff takes exactly $2|\mathsf{middle}| + 2$ transitions and keeps exactly $|\mathsf{middle}|$ cells throughout those transition states. A checked doubling-then-erasure run returns an empty word in thirty-four transitions but reaches four cells. Its peak is four, whereas the sum of component peaks is eight. This equality concerns transition states; the intra-command refinement remains a distinct measure.

Cook–Reckhow p-simulation conventionally compares proof systems for a fixed language such as propositional tautologies and requires a polynomial-time proof translator [1]. Our cross-logic comparisons also need the claim and theory translations. Reaching every recursively enumerable predicate with a suitably configured MM0 theory is not a proof that MM0 is a p-optimal proof system for tautologies. Different configured theories can assert different claims. Likewise, polynomial equivalence permits large constants and degrees; it does not imply equally short certificates or practical interchangeability.

5 Coupling proof search to proof checking

Fix a configured checker S, its physical certificate encoding, and a costed machine. A proof producer A is a partial algorithm which, on input j, may produce a certificate p. Its discovery

time $D_A(j)$ includes failed candidates, intermediate checks, indexing, scheduling, and the final validation. Soundness requires that every reported result is accepted by S. Completeness means that A eventually reports a result for every j with $\mathsf{Prov}_S(j)$. These properties do not require termination on other inputs. A finite budget returning no proof therefore returns a pending search state; it is not evidence of nonderivability.

Certificate and policy measures. For each claim, distinguish the least certificate length

$$L_S(j) = \min\{|p| : \mathsf{check}_S(j,p) = \mathsf{true}\}$$

from the checking work of a particular certificate, $V_S(j,p)$, and from $D_A(j)$ for a particular producer. An empty minimum is ∞. The nondominated pairs $(|p|, V_S(j,p))$ describe the certificate trade-off for j. They need not be computably identifiable: finding a better certificate does not certify the absence of an even better one. Code length of a *producer* is a fourth measure; it is not the length of the certificate that the producer eventually writes.

Proposition 4 (A bound which would decide derivability)**.** *Suppose certificates are finite words over a finite alphabet and checking terminates effectively on all inputs. If* Prov_S *is undecidable, there is no total computable function* $b(j)$ *such that every derivable* j *has an accepted certificate of length at most* $b(j)$*. Likewise, a sound and complete producer cannot have a total computable time bound* $t(j)$ *for all its successful searches.*

Proof. With the first bound, enumerate the finitely many words of length at most $b(j)$ and check them all. An accepted word decides the positive case, and their exhaustive rejection decides the negative case. All checks terminate. With the second bound, simulate the producer for $t(j)$ steps; a validated result decides the positive case, and the absence of one decides the negative case. Each construction contradicts undecidability. Neither argument asserts that every individual proof is long, or that every individual successful search is slow. □

Total checking is essential here. A partial verifier could accept a fixed one-bit certificate only after simulating a machine to termination. The certificate is short while the supposed checking operation carries the unresolved halting search. This is precisely why finite accepting runs and total checkers must be distinguished. The formal partial-recursive producer result uses the original `Code.evaln` evaluation budget. It proves that a computable budget sufficient for every positive query would decide the original meaning, and that increasing the budget preserves an accepted observation. This budget is not identified with a TM2 step count. Separate executed-search theorems use the original TM2 runs and charge their actual steps.

Bounded search and complexity classes. A declared length bound ℓ gives a finite exhaustive search with at most $\sum_{i=0}^{\ell} q^i$ candidates for an alphabet of size q. If every such check costs at most $v(j,\ell)$, its time is bounded by the number of candidates times that cost, plus enumeration and copying. This is an upper bound for that search policy, not a lower bound for every possible producer. A fixed claim language with polynomially short certificates and polynomial-time checking lies in NP; this hypothesis does not hold for unrestricted undecidable theoremhood. Checking a supplied proof in polynomial time is polynomial in *its input*, which includes the proof, not automatically in the claim length alone.

SAT makes the distinction concrete: one satisfying assignment witnesses satisfiability, whereas a tautology asserts truth under every Boolean valuation. Propositional completeness does not imply short proofs of every tautology. A polynomial-time proof system with polynomial-size proofs of all tautologies exists exactly when NP = coNP [1]. For an explicitly fixed finite domain, quantifiers can be expanded into finite conjunctions and disjunctions, potentially with large growth. This does not identify first-order or higher-order model expressiveness with propositional expressiveness on arbitrary or infinite domains.

Search transport requires executable witnesses. If a source producer returns p, certificate preservation gives a workflow producing a target proof of τj: retain the source query j, run its producer, compile p, and check $E(j, p)$. On a common sequential cost model, with accounted handoff cost h, its time is bounded by

$$D_A(j) + g(|j| + |p|) + h(j, p) + V_T(\tau j, E(j, p)).$$

Separate machine models require additional proved simulation overhead. This constructs one source-guided target policy. A native target producer receiving only τj additionally needs an effective query decoder and its accounted cost; neither follows from a computable forward map. It does not bound the fastest target policy or the shortest target proof. Conversely, global reflection $\mathsf{Prov}_T(\tau j) \Rightarrow \mathsf{Prov}_S(j)$ supplies no efficient source producer. A reverse search bound requires an executable certificate extractor, its acceptance theorem, and its cost. Enumerating source proofs until one appears gives eventual extraction under reflection, but no useful bound follows from that argument alone. Thus two derivability-equivalent presentations may retain very different proof search geometry. The implemented transport theorem also derives the produced-packet size from input size and discovery time by the stack-growth inequality. A producer cannot emit an enormous physical certificate within a small executed time bound. This implication does not assert that discovering a short certificate is cheap.

An observed budget is not a decision procedure. The finite-budget observer distinguishes a completed accepting run, a completed refusing run, and a still-live configuration. Its exact threshold is the number of original machine transitions, established from the original step function. For a witnessed producer, compiler and checker, the joined route has the executed upper bound

$$D + 2|\mathsf{sourcePacket}| + 2 + C + 2|\mathsf{targetPacket}| + 2 + K.$$

The two transfers are performed by the same physical composition used in the translation theory. Packet lengths can additionally be bounded from input length and the programs' actual writing capacity. A checked nonidentity route finishes in exactly seventeen transitions: a budget of sixteen leaves it live. An independent control accepts one certificate and refuses another for the same positive claim. Neither refusal nor a live observation refutes theoremhood. These observer theorems count simulated original transitions; an executed universal interpreter requires its own overhead theorem. Candidate enumeration is accounted for separately below.

The binary finite-search instance now supplies an actual bounded enumeration loop. Its depth budget is unary physical input; the current path, pending branches and every emitted candidate remain on finite-alphabet stacks. The enumerator follows the original finite certificate order, then transfers the full framed enumeration to the original checker loop. Its executed bound includes generation, both passes of the complete handoff, every candidate check, field copying and the final verdict map:

$$E(b) + 2|\mathsf{enumeration}(b)| + \sum_{p \in \mathsf{wordsUpTo}(2,b)} (K(p) + 2|p| + 4) + 14.$$

The result agrees with bounded certificate existence, and its finite-budget observation distinguishes completion from expiry. This construction charges refused candidates and the potentially exponential enumeration storage. It does not avoid materializing the enumeration or give polynomial time in a binary-coded depth bound.

A second executed route returns the original bounded search's first accepted certificate. It retains each actual candidate while running the original checker, frames the decision together

with that candidate, and then scans the resulting word for the first success. Its output tag distinguishes an accepted empty certificate from exhaustion. The selector drains the remaining input, so the clock charges every candidate check, the retained words, both physical transfers and final selection. Finite budget observation agrees with actual completion; readout recovers the original certificate and hence the configured checker's source meaning. This route still checks all candidates before selection. It establishes neither early stopping nor a universal scheduling bound.

A third executed route stops checker calls at the first accepted candidate. For rejected prefix P, first success p, and unread suffix U, its singleton-response caller clock is

$$\sum_{q \in P} (K(q) + 6|q| + 9) + K(p) + 7|p| + |\mathsf{frame}(U)| + 11.$$

The suffix is physically discarded, so it still costs bytes to clean up, but its checker executions need not terminate or even be supplied. An exhausted vector costs one final transition. A separate theorem joins this caller to the existing enumerator and charges $E(b)$, the complete handoff, and the caller clock. Thus early checking does not eliminate eager candidate-generation cost. Independent readout of any completed run returns the original bounded search result and inherits only the configured checker's supplied soundness. Eleven physical caller controls and their one-transition-short observations pass, including success before a later divergent candidate; divergence before success remains pending. The joined enumerator passes twelve complete executions and twelve one-transition-short observations. At depth five, the positive example finds the same one-bit certificate after 33 checker calls; exhaustive refusal takes 63. This preserves the existing depth-first order and makes its discovery cost visible even when the accepted certificate is short.

Universal scheduling links the two costs. Levin-style search schedules candidate *programs*, not just certificate strings. In a suitable prefix-coded machine and scheduling model, a candidate producer a receives weight comparable to $2^{-|a|}$. The search pays for generating its proposed witness *and verifying it*; its comparison bound has the characteristic factor $2^{|a|}$ times their combined work, with the scheduler and simulation overhead specified. A slow verifier can therefore dominate even when a candidate producer is fast. This does not decide negative instances or eliminate enormous constants [29].

Hutter's proof-guided algorithm selection is a further connection: it searches for proofs that programs compute the required function and that their execution has a stated time bound. Its guarantee is relative to the programs and bounds provable in the chosen background theory, and includes the time to compute those bounds and large proof-dependent additive terms [29]. This is not a uniform solution to theorem search and not a claim that there is a best unrestricted algorithm. The proofs authorizing a faster policy are themselves products of search, checking, and a declared logical authority.

Reuse and amortization. For a produced proof reused k times, write the whole cost as

$$\underbrace{D_A(j)}_{\text{discover once}} + \underbrace{T_E(j,p)}_{\text{compile once}} + \sum_{i=1}^{k} (H_i + V_i).$$

Initialization, reading, cache validation, and effectful work belong in the relevant terms. With unchanged authority and snapshot, a compile-once route with per-use checking cost v_T beats direct checking cost v_S only if its additional setup and translation cost is less than $k(v_S - v_T)$. If $v_T \geq v_S$, this checking-time argument gives no break-even point. Storage and peak space remain separate comparisons.

Positive and negative controls. Explicit equality and substitution witnesses can replace a checker's repeated normalization, provided a correctness and cost theorem justifies the replacement. Verified caching can reuse a checked shared lemma. In contrast, merely delaying a producer or verifier by a computable amount changes discovery or checking time without changing accepted claims or certificate lengths. A one-off translation can also cost more than the checking work it saves. These examples separate the resource measures; they establish no performance claim for a particular backend.

6 The explicit base and computational compression

An explicit base checks supplied proof-relevant steps. It still parses, compares terms, follows references, performs substitution, and validates scope. MM0's explicit design does not mean that its implementation does no computation [4]. HOL Light's equality rules similarly expose inference steps while implementing term transformations.

For a deterministic effective checker, a finite accepting execution can be supplied as a trace. A checker of fully written consecutive configurations can validate the trace without rediscovering the run. This motivates the general explicitization theorem: host a finite presentation of the machine's transition relation, prove its trace judgment adequate, and compile the trace certificate. Such a theorem must include the accepting configuration and exact statement linkage; merely proving a machine can halt is insufficient.

The resulting certificate can be polynomial in the *written trace* and nevertheless enormous relative to the original proof. This is how a trace-explicit base can be operationally universal without erasing resource differences. Composition of elaborators also collapses arbitrary counts of intermediate layers; raw elaboration depth cannot alone define a strict hierarchy.

The executed interface also admits non-polynomial per-packet clocks. These translations compose through the same physical handoff and can feed an exact target checker. A proved polynomial clock recovers the polynomial interface; its output-size bound follows from actual writing work. Both relations are preorders under the same replay/reflection policy, and every polynomial route is an executed route. A loose upper bound is not a machine lower bound: separation needs an independent argument such as unavoidable output growth.

A checked normalization separation. In the existing Mathdata model, set $u_0 = a$ and $u_{n+1} = (\lambda x.\, f\, x\, x)\, u_n$, with fixed well-typed primitives. The bottom-up normalizer produces the full binary tree v_n in one changing pass, followed by a stable pass. The original instrumented elaborator records exactly n beta contractions. Nevertheless, the existing finite-alphabet term codec gives the exact equations

$$|\mathsf{encode}(u_n)| = 1244n + 305, \qquad |\mathsf{encode}(v_n)| + 429 = 734 \cdot 2^n.$$

The executed output-growth law therefore excludes every polynomial-time machine emitting this full expanded representation from these inputs. Pass fuel and contraction count do not measure the cost of traversing, copying, or writing its payload. This is a theorem about expanded output, not an inherent lower bound for Boolean conversion checking or a shared output representation, and not a verification of source Megalodon.

The same family now has a checked shared representation using the original chronological Pattern table. Four initial nodes and two application nodes per level suffice: there are exactly $2n + 4$ nodes, and complete binary serialization, including reference addresses, has length at most $512(n + 1)^2$. The existing materializer reconstructs precisely the original encoded v_n. Future children, future roots, and trailing symbols are refused. At $n = 8$, the expanded wire contains 187,475 symbols and the shared wire 1,684 symbols. Their size ratio is unbounded. Emitting the full tree serialization from these shared packets still admits no polynomial-time

machine. This lower bound concerns physical expanded output; it does not assert that the materializer's heap representation, graph checking, or Boolean conversion must expand that tree. The executed source-to-shared compiler remains a separate obligation.

Positive example. A saved lemma referenced twice needs one checked initializer and two references. A graph-preserving compiler retains that sharing.

Negative example. Inlining a shared proof recursively can produce an exponential tree. A bound in graph nodes is also insufficient if each node carries a large term, a large binary reference, or a large context.

Equations. An explicit equality certificate can justify reflexivity, congruence, transitivity, substitution, and individual definition-unfolding steps at the base. Implicit normalization shifts proof-relevant work to checking. Whether the result admits a polynomial explicitization is a theorem about that conversion procedure, not a consequence of calling it conversion. Unrestricted normalization can be costly [14]; sharing and the chosen reduction strategy matter to a reasonable cost model [15].

Adding arbitrary equations as axioms can change the derivable claims, and can make a theory inconsistent. Replacing a derivable explicit equality by a proved equivalent conversion mechanism need not add claims. Both theories can remain recursively enumerable while proving different statements. Recursive enumerability is not sameness of logical authority.

7 Models, higher-order syntax, and binders

The semantic comparison should use independently defined models and satisfaction. For a model translation $R : \mathsf{Mod}(T) \to \mathsf{Mod}(S)$, the institution-style condition is

$$M \models_T \tau(j) \quad \Longleftrightarrow \quad R(M) \models_S j.$$

Signatures, sentence translations, and contravariant model reducts must commute with renaming and imports [5]. A certificate simulation alone says none of this.

Henkin models restrict function domains while maintaining the closures required by the chosen typed language and its terms. Full models use full function spaces. Henkin completeness is a semantic theorem for a specified calculus [6]; it does not follow just because another checker accepts that calculus's proofs. Sufficiently expressive full second-order validity is not recursively enumerable, so no effective checker is both sound and complete for it [7]. An effective calculus can still be sound for full semantics. This restriction concerns full validity, not every restricted fragment or every individual full-model theorem.

HOL's kernel can therefore have a first-order *presentation of its checking judgments*, while the object logic has higher-order types and quantification. Explicit substitutions provide direct precedent for such presentations [8]. That is not a claim that full HOL semantics has become first-order semantics.

Binding alone does not require a computational tier. Names, de Bruijn indices, nominal syntax, or a scoped binding signature can be finite data. The substantive obligation is adequacy of typing, alpha-equivalence, freshness, capture avoidance, and substitution under that representation. Generic binding signatures and second-order metavariables offer reusable abstractions [10, 11]; declaring a binder does not automatically prove the adequacy laws. Higher-order unification or implicit equality solving introduces a further algorithmic obligation.

For HOL-to-MM0, one may encode binders as first-order data or use MM0's native binder discipline. For calculus-to-HOL, one may encode syntax as data or use host lambda binding.

These choices require distinct adequacy theorems. No ABT substitution node or particular runtime representation is imposed as the definition of the intended general framework.

Conservative extensions. Lifting a definition into an explicitly listed axiom records an honest theory dependency. It is not a proof that the enlarged theory is conservative. Model expansion, consistency, and preservation of old derivable judgments differ; model conservativity does not alone imply proof conservativity without appropriate completeness [12]. Whole polymorphic histories of definitions and type definitions need their own transport and extension theorem [13].

Finite evidence for standard infinite meanings. The atlas now has two exact, deliberately scoped examples. `PresburgerUniversal` structurally normalizes Mathlib's existing Presburger terms to affine forms. It decides universally quantified, hypothesis-free affine inequalities over the ordinary natural numbers. Acceptance is equivalent to truth for every natural valuation; refusal is equivalent to existence of an actual countervaluation. This does not decide arbitrary Boolean combinations or quantifier alternation.

`CertificateGSLTRootBuchiAuthority` checks a finite invariant region and progress ranks for a fixed controller on a finite carrier. Existence of accepted evidence is equivalent to legal admitted execution from the selected root with accepting states visited beyond every finite prefix. The invariant region excludes unreachable bad components, which would otherwise obstruct completeness for a root-specific claim. `StandardSemanticAuthorityProfiles` places the arithmetic and recurrent claims under one exact authority family and projects checked edge witnesses to genuine infinite GSLT execution. Its negative controls separate a winning selected controller from a nonrecurrent alternative path, and a real path from invalid supplied evidence. These are not S1S, S2S or real-closed-field decision procedures.

The unrestricted ω-rule instead requires the whole infinite family of premises. Finite induction or a certified cyclic argument may establish particular universal claims, but does not validate every purported infinite proof. For the recursive proof encodings studied by Frittaion, recognizing valid ω-proof codes is Π^1_1-complete [28]. This lies beyond the finite accepting-run boundary of ordinary partial checking; the finite recurrence certificates above certify a restricted infinite meaning.

8 Lean and further computational kernels

Lean is not merely HOL with faster proof compression. Its dependent types, universes, inductive families, quotient machinery, and configured axioms also change the language and its foundational commitments. To compare logical authority, first state the claim interpretation and the assumptions under which it is sound. To compare compression, keep that interpretation fixed and then compare certificates and checker work.

The Lean reference describes beta, delta, iota, zeta, and quotient reduction, eta principles, and definitional proof irrelevance [18]. These should become explicit, separately admitted conversion profiles in a hosting construction. Reduction rules, the implemented conversion algorithm, and axioms such as classical choice must not be amalgamated into one unexplained "higher tier." The independent Lean4Lean checker and its partial correctness development are relevant inputs, not a finished verification of the entire current kernel [19].

The implemented algorithm and its resource limits require versioned source claims. In Lean 4.33.1, `CoreM.maxHeartbeats` counts small allocations in thousands; it does not count beta reductions. The kernel type checker has system and recursion-depth guards in its inference and reduction paths [27]. The generic bounded-evaluation results above do not identify their bounds with these counters or verify this entire C++ kernel. Undecidability of an

ideal equality relation, incompleteness of an implemented recognizer, exhaustion, and actual divergence must be established separately.

There are two possible statements with different acceptance criteria. A *run-replay theorem* turns an actual accepting run of the specified implementation into an explicit accepted certificate. A *declarative adequacy theorem* identifies the accepted judgments with the intended dependent type theory. Replay alone cannot exclude implementation bugs in the algorithm being replayed. Conversely, declarative soundness does not bound the runtime of a particular implementation.

Normalization and decidable conversion also need careful hypotheses. Proof irrelevance does not by itself determine either result: closely related systems have negative normalization results and positive reduction-free normalization arguments [20, 21]. We should isolate the precise fragment, equality principles, universe rules, and algorithm rather than transfer a result by resemblance to Lean.

The current implementation separates these cases more finely:

Profile	Established comparison	Remaining qualification
MM0	Original binary assertion/file acceptance yields original kernel derivability, under decoded declaration correspondence. Original quoted reference/save/hypothesis runs preserve metadata and refusal; newly emitted hypotheses reach the original proof checker. The original theorem resolver validates ordered typed child proofs, with exact chronological lookup and inert recording.	Complete native binary proof-run correspondence and complete executed translation bounds.
HOL Light	Configured MM0 acceptance connects to original proof reachability, native-type observation, and extensional Henkin-model truth with explicit axiom-schema obligations; admitted constant histories construct a final model preserving all earlier type instances. An original admitted type definition has a model of both retained equations at each fixed instantiated definition.	Original kernel-history/schema transport and executed end-to-end comparison routes.
Mathdata	The modeled beta/eta/delta normalizer emits explicit accepted conversion articles; a typed family has linear encoded input and exponential expanded output, with polynomial shared storage and no polynomial full-tree emission.	Source Megalodon's lazy conversion and beta counter are different algorithms; the model's pass fuel is not their runtime.
Lean fragments	Independent beta-head and finite-catalog delta executors; typed reduction correspondence when typing evidence is supplied.	Full typing, equality, inductives, quotients and universe admission are not obtained from the raw-step executors.

The Lean typed-boundary control accepts a raw beta step in an environment where its argument constant has no type. The corresponding typed native observation rejects it. This makes the distinction between a correct raw reduction engine and a qualified typed kernel

concrete. The more general `Lean4LessCheckerBoundary` structures state obligations; they are not instances establishing verification of Lean's whole kernel.

There are three distinct directions "beyond Lean." First, a configured checker may perform additional deterministic computation, certified decision procedures, or reflection while preserving a specified judgment relation. The questions are its executed time/space and the cost of writing an explicit certificate. Second, additional universes, axioms, or reflection principles can enlarge logical authority; this requires an interpretation and its metatheoretic assumptions, independently of certificate compression. Third, full higher-order validity can serve as a semantic specification beyond effective complete checking. It is not another executable tier. These choices do not entail a strict ordering of particular implemented systems. Interactive and probabilistic verification would additionally require a protocol and error model, outside the present deterministic interfaces.

Further kernels, including rewriting frameworks and verified reflective checkers, can be added as configurations and arrows in the same graph. Each arrow retains its authority, representation, and cost conditions. There is no requirement that every pair be directly comparable, or that stronger assumptions always buy smaller proofs. Bootstrapping then means checking particular implementation-correctness claims under explicit metatheoretic assumptions, not making a system prove its own soundness without an external justification.

9 Modules and operational native types

A `.module`/LanguageDef surface can be the single authoring source for syntax, rules, named imports, and interpretations. Backend emissions then present views of the same admitted theory. An MM0 printer and an OpenTheory exporter should share declaration identities, dependency ordering, scope, and checked module interpretations rather than create parallel theory authorities. LF's adequacy discipline remains useful when describing these representations [9].

There are two separate composition laws to prove: translation of a compatible module join agrees with joining translated modules up to an explicit renaming; and interpretation transport preserves the declared derivations. Meet, replacement, definitions, and recursive imports do not inherit these laws merely from notation. Conflict rejection and future-reference refusal remain observable behavior.

For GSLT-to-MeTTa execution, the compared objects are operational systems as well as checkers. The source and target property languages and evidence families must be defined independently. Matching rule occurrences is only the first obligation: premises, typed substitution, permissions, contexts, equation classes, effects, and revisioned providers enter the source-visible execution relation.

Finite reachability transport can use a source step represented by a nonempty target path. A source-visible modal contract hides administrative target steps at declared boundaries. Universal, normal-form, and divergence-sensitive properties require more: internal progress or termination, treatment of branching, and the appropriate backward coverage law. These are stronger than acceptance preservation.

Truth-valued observations can miss two identical results carried by two different rule occurrences. Evidence families retain positions, bindings, provenance, order, and multiplicity. An effectful failed branch can also retain output. A proof of "at most one answer" does not license discarding an executed printing branch. A bounded execution should return remaining obligations rather than assert that no result exists.

9.1 Presentations, formation, and conversion

A local native-type cube is indexed by a presentation and a displayed operational site. A machine presentation can hide equation lookup behind a premise, whereas a program presentation exposes a callable head inside a context. The two interfaces can consequently have different slots. An interpretation between these presentations must earn its operational and native-type transport laws; neither an equal cube nor an equal set of accepted claims supplies those laws. Conversely, faithful semantic transport need not identify the slot sets or select one uniquely canonical presentation.

The proved slot-count comparison in `SiteInterfaces` counts index variables in addition to the rely-and-output slots of `Slots.modalFamily`. For the displayed MM0 application site, these are distinct interface descriptions. Their vertex counts must not be substituted for one another. Target premises, output carriers, and the treatment of equations also require a stated interface convention. The bare completion's lack of closed classifiers is a property of that completion; it does not mean that the object language lacks meaning or that its proof checker is unsound. Constructor classifiers, closed classifier inhabitation, and inhabitants of a classifier are separate questions.

The source module `TypeTheory.HypercubeBubbles` independently parameterizes product formation and conversion. Its separation results exhibit a polymorphic judgment requiring a formation rule that no added conversion supplies, and a conversion judgment that increasing the formation vertex alone does not supply. These results concern their explicit pure-type-system presentation. An NIK profile therefore records its presentation, formation rules, conversion policy, logical authority, and executed resource grade separately; a vertex is not a numeric verification tier.

For a proposed presentation bridge, the remaining obligations include the source and target carriers, legal substitutions, administrative-step boundaries, correspondence of indexed sites, preservation and reflection of the chosen observations, evidence transport, and the execution cost of these maps. Forward possibility, the predecessor-quantifying right adjoint of OSLF's diamond, and the forward universal-successor modality must have distinct contracts. A finite search's silence licenses none of their negative claims without the corresponding completeness witness. The cited interface and formation/conversion sources have been inspected; their recent changes are not included in this paper's joined resource checkpoint or silently imported into its qualification claim.

9.2 Independent source checkers and optional typed representations

The foundation here consists of independently specified source judgments and checkers for a guarded atom-rule calculus, the GSLT-metalanguage command calculus, and a rho profile. Their mathematical validity does not depend on choosing a Prime candidate. A representation in another calculus is a separate comparison result: it neither defines the source checker nor completes its implementation. Prime remains an unsettled family of design candidates.

Ownership follows the hypotheses of each construction. Generic service qualification and interface matching live in `SemanticService` and `SemanticServiceFamily`; demand and resumption laws live with machines. Source admission, binding and replay remain with their existing GSLT or language owners. Pattern codecs for the parameterized and cumulative Pi/Sigma/Id calculi live with those calculi. Candidate-specific representations and comparison controls consume this mathematics downstream.

There are three different implementation claims to distinguish: evidence native to a source calculus, a checker written in a MeTTa dialect, and a compiled native executable. A result about one does not establish the other two. Primitive checking discipline and executed resource class also remain separate obligations.

9.2.1 Three explicit replay formats

The checkers retain different evidence structures:

Profile	Evidence	Independent source judgment
Guarded atom rules	Named Atom rule rows, intermediate assignments and earlier-row references	Conditional root firing, guards, queries and callable readiness
GSLT-ML	`at`/`via` journals with local fibre or transport evidence	Computation within a fibre and the selected explicit transport
ρ	Original inference trees and COMM computation call trees	Structural congruence, name equivalence, reduction, reachability and permutation

For each profile, validity of the supplied article is specified independently of its Boolean checker. Exactness concerns that article, including repeated premises and intermediate data. Coverage constructs accepted evidence for every finite derivation in the stated source family; it does not decide whether a derivation exists.

`GuardedAtomRule.Evidence` uses the existing conditional Atom source judgments and its three registered freshness and matching services. Its executable is `guardedAtomRuleCheck`. It checks guarded atom-rule firing; it does not check HE or PeTTa evaluation. Its authored `rule!`, `relation!`, and `builtin!` forms belong to that calculus; they are not established MeTTa dialect constructs. In particular, root firing alone does not provide nested argument evaluation. Faithfulness to a MeTTa dialect requires correspondence with that dialect's independently specified evaluation judgments. Its `ArticleValid` requires every supplied row, including unused rows, to be justified against its earlier prefix, and requires the exact root judgment. Canonical Atom admission and proof validity have separate equivalences. Executable expansion is complete for original root firing and callable readiness, retaining the original call body separately from its assignment. The declaration authority is the complete submitted source row list, rather than an unstated mutable world. The expanded replay presentation has 57 primitive cases and 19 source cases. Six name-inequality cases expand a fixed string codec into bit evidence; this is a representation choice, not a theorem that MeTTa requires that primitive basis.

`GSLTILNativeKernel` replays the three original command rules using separately qualified local evidence. Its concrete finite-catalog checker covers exactly the original wire steps and finite paths, retaining row ordinals and complete intermediate states. A second concrete instance has an unbounded Tick fibre and a proved obstruction to covering it by a finite catalog of concrete edges. The generic interface does not establish an effective or inexpensive checker for every possible fibre. Structural packet decoding is distinct from parsing arbitrary authored `.module` text. Successful replay requests each journal occurrence exactly once; indexed lookup visits exactly the minimum of the requested prefix length and the catalog length.

Checking a rewrite inside an authored language. `GeneralRewriteRecipe` supplies the previously separate local rewrite layer. It checks the actual selected rule or equation, a licensed typed occurrence, scoped matching choices, and original premise evidence. The public boundary compiles the submitted language snapshot and permissions; a caller-supplied execution plan is not declaration authority. Guided matching follows the recorded bag selections instead of enumerating all permutations. Guided premise checking similarly follows the supplied finite query and child evidence. Exactness is with respect to the pre-existing occurrence and premise judgments, rather than a new relation defined to match the checker.

A primitive firing may have equation conversions on both sides. Every such accepted expanded step trace justifies the original equation-saturated step, and every finite derivation of that step has an accepted trace. The theorem covers the admitted authored presentation; it does not assert effective checking for an arbitrary mathematical relation called a GSLT. The public occurrence instance uses the empty dynamic provider. The lower premise checker also supports independently specified dynamic services through their explicit correctness contracts.

`GeneralRewriteRecipeExpansion` distinguishes supplied choices from expanded evidence. A `Recipe` retains the rule position, typed address, matching choices and premise evidence. Replay computes the assignments, zipper and result. Every valid original firing is reconstructed exactly from this projection. A `StepTrace` additionally retains the computed firing and surrounding equation conversions. A receipt is a record of an application; being recorded does not itself establish validity. The shared `GeneralOccurrenceKeys.FiringIdentity` retains the rule, typed site, assignments and premise citations. Its citation projection forgets matcher permutations and physical positions of quantified bag elements recursively, while retaining multiplicity and source/provider citations. Equality of valid firing identities preserves the result. Lossless `RuleReplayRecord` and `EquationReplayRecord` retain the full replay guidance separately. Thus equal bag entries need not acquire different application identities merely from selecting different positions.

`GeneralRewriteRecipeComposition` checks sequential joins. It also checks equation-respecting semantic paths. Concatenation retains both equation histories at the join and adds the number of primitive firings. The accepted paths characterize the existing existential finite-reachability native types, including their exact primitive grade. A `RecipePlan` is a sequence with some recipes still missing. Its execution stays pending until all choices are supplied; a complete plan is then either refused or produces a checked trace with computed intermediate states. A separate `TracePlan` has holes in already expanded evidence. Sequence, graph and parallel arrangement are distinct from the question of whether all choices have been supplied. Empty paths express reflexivity on the source carrier; alone they do not certify formation, termination or a property of every execution.

`GeneralRewriteRecipeDAG` admits chronological references to earlier endpoint judgments. Every declared node is checked, including unused nodes. Its expansion preserves each reference use and the original firing payloads; using one saved proof twice does not turn two uses into one. The graph checker is exact for its independently stated node rules, and accepted graphs cover exactly the existing finite semantic paths. Canonical wire formats distinguish expanded atomic step traces, literal traces, semantic traces and DAGs. The executable `gsltRewriteCheck` accepts those structural formats, not authored `.module` text.

`GeneralRewriteRecipeContext` composes licensed typed sites and lifts original rule and equation evidence through them. It preserves the local assignments and premise histories, proves the exact enclosing result, and agrees with the original completion functions. Local binder and provider contexts must match; enclosure does not silently rebase those contexts. `GeneralRewriteRecipeParallel` checks a supplied square of four primitive firings. Prefix-incomparable addresses, stable site labels and literal endpoint closure construct the existing independence tile and Mazurkiewicz swap. It neither searches for residuals nor asserts that every pair of rewrites at separate sites commutes.

`GSLTILAuthoredFibre` installs this actual authored rewrite relation as the local step at an explicitly selected stage of the existing command checker. Its checking equivalence and source coverage are proved, including steps underneath a pending route frame. This instance installs no transport routes and makes no interpretation into a language candidate.

Local replacement preservation connects the licensed hole, actual parameter representation and locally admitted result to the whole target's typing, scope and canonical metadata. The rule checker uses this theorem to avoid a redundant whole-target sort check. The separate expansion function still uses the original completion routine and its target check. A wire

boundary retains all original fields, rejects aliases and trailing input, and validates against the compiled authored snapshot. These results establish exact replay and finite-evidence coverage. Primitive substitution, sort checking and header compilation still have their own costs; a polynomial bound in serialized input size is not claimed here.

`RhoCalculus.NativeKernel.Explicit` checks all 33 original raw rule forms. A COMM node supplies a literal contractum and a finite call tree for the existing name normalization and communication substitution. Replay checks local constructor cases and child results rather than running the original recursive substitution as an implicit inference. The validity equivalence retains the same call tree and every child occurrence; a bare claimed result is refused. Every original finite raw derivation has such explicit evidence. Optional endpoint admission checks the original Proc/Name formation and a separately stated quotation-scope condition. It neither narrows raw coverage silently nor identifies this source with the different scoped twelve-rule conversion calculus.

`RhoCalculus.ScopedNativeServices` separately owns the original intrinsically scoped source operations. Its terms use the existing two-sort rho binding signature. Equality is the original generated parallel ACU and name QuoteDrop theory; directed COMM and process Drop use the original quotient-event observation and parallel-only contexts. The source packet checker carries contexts, sorts, terms and equality or step evidence without target Data. Its executable is `rhoNativeServices`; the optional typed representation has the separate `rhoCandidateServices` client. Generic binding-equality certificates are reused here; they are distinct from the optional twelve-rule rho proof-tree presentation.

The guarded atom-rule structural work bound applies after literal endpoint-word materialization and rule admission. The GSLT-ML counts concern actual local requests and list-cell visits. These are useful resource accounts with different observers; neither establishes a physical clock for a complete serialized client. The executable clients are compiled Lean references. Source-native evidence, a checker written in the source language, and a verified native compiler remain separate properties.

9.2.2 Independent evidence and source operations

The common finite-rule construction now connects the existing `JudgmentDerivation` family, indexed rule polynomials, native certificate trees, and an untrusted list-tree wire. The cartesian rule map retains each recursive occurrence, and its action commutes with the independently defined derivation translation. The raw decoder rejects missing and extra children; `Tree.derivationWireEquiv` identifies its accepted fibre with the original derivation fibre. `FiniteRuleKernel.wireCertificateEquivalence` then connects that fibre to the existing native-service interface. None of these results treats an arbitrary target proof as an image proof: the extra-rule control has a target theorem absent in the source, even with an injective judgment map.

For any existing binding signature and finite equation list, the constructed equality replay uses finite vectors for simultaneous substitution and tables for contextual metavariable bodies. It proves

$$\mathsf{EqClosure}_E(t,u) \quad \Longleftrightarrow \quad \exists p.\ \mathsf{check}_E(p,t,u) = \mathsf{true}.$$

The left-hand side is the pre-existing generated equational theory. Completeness constructs finite evidence from each of its derivations, including congruence under complete binder spines. This is not an equality decision procedure or a normalization oracle. A control distinguishes capture avoidance from a captured term even though both candidate outputs have valid scopes and the same sort; another refuses transitivity with unequal intermediate terms. The raw equality boundary now decodes scopes, variable indices, operator arities, complete binder arguments and retained equation indices. Its roundtrip and acceptance theorems connect

those packets to the same generated equational theory. Explicit substitution nodes can nevertheless compress large intermediate expansions. Neither finite evidence nor exact replay alone establishes polynomial checking in the submitted byte length.

The concrete profiles keep their source judgments independent. The rho checker covers the existing raw-pattern structural congruence, name equivalence, reduction, reachability and permutation relations; formation is a separate admission. Its complete structural storage retains all 33 rule-instance forms, the requested goal and every recursive occurrence. Canonical whole-word acceptance constructs an original source derivation, and every original derivation has an accepted request. A linked executable exercises that same checker through files; this does not verify its native compiler toolchain.

Shared structural evidence uses the existing chronological certificate graph. Each row is checked, including unused rows, before its claimed judgment enters the saved table. References retain their source occurrence and multiplicity; sharing a proof does not identify distinct uses of it. The shared rho instance uses the separate scoped intrinsic conversion profile with twelve rule forms. It has an exact graph-checking correspondence for that family. It is not a shared implementation of the preceding raw-pattern checker with its thirty-three rule forms. Its instrumented judgment-cache routine performs at most $2N+1$ requested judgment comparisons for N rows. This count excludes the cost of obtaining referenced rows and checking each rule instance. The separate binary equality machine charges the compared words; neither result alone is a clock for the whole serialized checker. Disjoint literal packet headers route the original tree requests and shared graph requests, preserving the original accepted bytes and refusals.

Presentation morphisms also act constructively on these shared graphs. The map retains every row, reference ordinal, repeated premise use and root; it does not expand a graph into a tree. Successful checking is preserved without injectivity. With an injective judgment map, the complete checking result, refusals and instrumented row/premise counts agree on the image. Identity and composition laws hold for the actual graph maps. A collapsing judgment map supplies a negative control for reflection. These counts exclude judgment equality, payload construction and physical byte costs.

Finite evidence for conditional authored rules. `GeneralPremiseRecipeChecking` and `GeneralRewriteRecipe` validate the original occurrence receipt, including its selected declaration, scoped matcher trace, captured and completed assignments, and nested premise events. The existing source judgments specify root firing, individual premises, premise sequences and quantified elements independently of the checker. A finite receipt can cite a recursive child at any declaration index: a decreasing declaration number is not required. Freshness, static or native query origins, contextual recursive steps and quantified collection elements retain their original side conditions.

The validator follows the supplied bag choices, query citations and child evidence. It computes substitutions and checks the original premise relations without enumerating every matcher answer and then filtering them. Structural recursion on the submitted finite evidence gives a total checker with

$$\mathsf{checkFiringWith}(\Delta, \Theta, \Gamma, A, s, p) = \mathsf{true} \quad \Longleftrightarrow \quad p.\mathsf{ValidWith}(\Delta, \Theta, \Gamma, A, s).$$

Here Δ is the fixed dynamic provider, Θ the original admitted plan, and Γ, A the full source context and result type. The corresponding `GeneralPremiseNativeEvidence` service fixes both source and requested target, and accepts exactly these original receipts. Its judgment is stated independently of its Boolean implementation. Totality is not a physical-time bound: preprocessing, assignment comparisons, copying and provider computations require their own accounts. Total validation of a supplied finite receipt also does not decide whether a receipt exists.

`Comparisons.EnumeratingPremiseReplay` retains the older enumerate-then-filter implementation as a comparison. Its cost controls produce $7! = 5040$ matcher answers for seven bag entries where guided replay follows one supplied selection. Seven equal entries likewise produce 5040 branches with one distinct assignment. No source checker or native modal introduction depends on this comparison implementation.

The full original plan, job, assignment, event and receipt have canonical finite-word codecs in a candidate-independent source owner. The validator and discovery service do not require a Pi/Sigma/Id interpretation. The source client `authoredPremises` calls the source-owned packet engine directly. The separate candidate adapter attaches its target Data only downstream. Source accepted data does not by itself construct a dependent proof inhabitant. The source request and proof headers are `GSLT.AuthoredFiring.Request.v1` and `GSLT.AuthoredFiring.Proof.v1`; source firing and discovery services likewise use `GSLT.AuthoredFiring.v1` and `GSLT.AuthoredDiscovery.v1`, with distinct reply headers. The former candidate-branded source headers are not accepted as aliases. Target-only declaration names remain explicit representation choices.

Retained proof discovery interleaves the existing source runner with an exhaustive finite-word fallback. Every proposal is rechecked by the original total validator, and every valid finite original receipt eventually enters the journal. Whole-article deduplication retains distinct source proofs of one claim and does not remove occurrences inside an article. Exact budget composition and reachable-checkpoint resumption are proved. This observation concerns discovery of proof articles, not operational event multiplicity. The exhaustive index bound may be enormous; no efficient-search or physical time bound follows. Arbitrary serialized checkpoints require an additional admission theorem.

The finite revisioned MeTTa profile checks actual source query articles and persistent-emission receipts and realizes their composition with admitted ground or support-aware contextual continuations. It preserves occurrence multiplicity and causal state changes without imposing an order on language-level answers. Its current occurrence checker may enumerate a matcher bag; source exactness is not a polynomial resource bound for that enumeration. The original effectful Atom machine additionally has a finite step and path checker covering named-space and cell changes, allocations, completed answer bags and exact budget composition. This proved component uses a pinned program index. Storing a fresh equation does not by itself activate it in that index; live reindexing and guarded code-entry activation are separate operational policies. The constructed `MeTTaAuthoredProgramExecution` runner interleaves the original Atom evaluator with fair conditional-call search. Every admitted finite source article is eventually discovered; an unfinished search remains unknown. A call may explicitly enter under its pinned source snapshot, or require literal agreement with the current world. These policies differ after mutation, and stale current-world evidence cannot authorize entry. The runner retains actual evaluator frames, causal body effects and exact budget composition. Independently stated source transitions now preserve and reflect each conditional controller step. Ordinary work uses the original Atom transition judgment, while guarded entry requires original source evidence. Completed reachable search has exact finite source-article membership coverage; this is not an article-to-event uniqueness theorem.

The program layer handles all fourteen existing command forms and provided module imports using the original first-match module lookup. Its text boundary shares the original command classifier with the existing parser while reading directly into Atoms, preserving the distinction between a symbol and a nullary expression. Guarded declarations retain their premises rather than becoming unconditional equations. The reader theorem is relative to the original scanner and expression reader; it is not an adequacy theorem for an arbitrary grammar presented as data. The maintained executable uses this same runner, with distinct completed, refused, stale, exhausted and unknown outcomes. Its command-line module uses the source-owned guarded atom-rule service directly. Optional candidate representations are

separate consumers. Independent whole-program source transitions also preserve and reflect the fourteen-command runner, including import return and causal effects. Finite source action chains realize, and are recovered from, actual bounded execution. Their lengths charge the selected controller's administrative steps; stopping before the supplied budget requires a genuinely stopped source state. These are action counts, not physical running-time bounds.

The original compressed Atom proof checker has a concrete materialization boundary. One fixed source theory and fixed final claim admit articles of size $5n + 10$, with canonical raw-Atom structural size affine in n. Their final freshness check uses the original free-variable routine, whose materialized occurrence list has length exactly 2^n. No fixed polynomial in either article-size measure bounds that list. This does not establish a physical clock or rule out a different efficient checker. It explains why this compressed checker cannot inherit an expanded-work bound without accounting for its generated intermediate data.

These are explicitly specified Atom calculi and a conditional extension with its administrative controller. Their relationship to a MeTTa dialect requires a separate adequacy theorem; root rule firing alone is not dialect evaluation. A compiled Lean client is also distinct from a checker written in and executed by either MeTTa dialect. An expanded-evidence checker needs its own construction and resource proof; the compressed checker's source exactness does not supply that proof.

GSLT-metalanguage scope. The `NativeCommand` service in `GSLTILNativeKernel` checks indexed source instructions and constructs occurrence-indexed successors for its finite catalog of literal endpoint rows. Its source judgments, checker and execution laws have no candidate dependencies. The downstream `FiniteCatalogInterpretation` is explicitly a comparison with one specified calculus. That finite catalog is not the full GSLT metalanguage with arbitrary binders, conditional premises, equations and module commands. The indexed operational GSLT-IL syntax and free-path semantics, and the authored-rule recipe checker, remain distinct owners. The authored-fibre instance above connects actual rewriting to the command checker at its stated scope, without extending the finite catalog into a universal semantics for module commands or transports.

HE's static typing contract is a useful independent backend interface. Its translation preserves flat-arrow structure using explicit type operators and application constants, rather than silently replacing every HE arrow by a curried HOL function type. Gradual unknowns and metatype-directed evaluation retain their own meanings. Agreement with this contract does not by itself bound complete type enumeration or type-directed program evaluation. Resource claims must include the admitted type environment, branching, generated output, and the executed conversion or evaluation policy.

9.2.3 Optional representations in a declared Pi/Sigma/Id calculus

The following constructions concern one explicitly specified target calculus. They are reusable interpretation, binding and conversion mathematics. They are not a choice of Prime, a common foundation required by the independent source checkers, or completion criteria for the three native kernels.

As an optional target-specific construction, the scoped intrinsic rho profile has a structural encoding into the declared Pi/Sigma/Id calculus: variables remain variables, input bodies become target lambdas and six distinct declared constructors carry the source operations. Renaming and simultaneous substitution commute with this encoding. Formed target declarations supply actual typing judgments. A separately defined, context-and-sort-indexed native structural relation preserves and reflects the source equations on the full image. Its quotient embedding is injective and commutes with substitution. Constructed target COMM and Drop operations, closed only under parallel frames, similarly preserve and reflect the

selected source events. In particular, process Drop executes but is not a structural equation. The raw communication checker contracts a quotation/drop roundtrip during semantic substitution; the intrinsic rule retains that residual. Their general adapter therefore needs a stated guard. The constructed raw adapter retains closed, quotation-safe admission and uses a separate residual equivalence where the source substitution policy requires it. The routed native service checks these guards and keeps equation, operation and proof roles distinct. It does not identify every raw pattern with an intrinsic process, or claim reflection of unrestricted ambient type-theoretic conversion.

A separate binding-signature interpretation constructs target operators as closed lambda programs. A fold of the original free binding terms preserves renaming and simultaneous substitution; a primitive operator's actual native typing yields the whole term's typing by induction. Contextual schema programs retain the original metavariable assignment and explicit dependence list. Generated finite beta evidence connects their emitted programs to the fold of the original source instantiation, including ambient variables and nested substitution. Formation is proved before execution, using actual dependent function types rather than uninstalled polymorphic identity declarations.

Native function parameters make such programs reusable. Simultaneous interpretation acts on primitive constants and source variables, lifting both under every binder. For root-free source templates, primitive declaration typing, an independently formed parameter telescope and a typed argument substitution imply formation of the resulting closed program and its application. Distinct actual projection functions instantiate the same program to distinct results. The generated application certificate selects exactly one beta step per parameter. This count excludes payload copying, endpoint comparisons and implementation instructions; it is not an executed time bound. A single schema template is now constructed in empty ambient context with finite metafunction constants. Open interpretation of those constants by the actual source bodies, partially applied to the caller's ambient environment, yields the original compiler output at arbitrary ambient contexts and under nested binders. The generated evidence still replays to the independently specified source instance. This result does not quantify over arbitrary native functions as though they were source bodies. The installed conditional rule family and its original premise authority remain additional obligations.

Named-output restoration covers the original successful raw instantiation, including substitution under binders and collection rest captures. Per-site capture views preserve original declaration ordinals and use the actual input/output modes: an output variable need not be present before the query binds it. Lexical shadowing and restoration retain their original scope. Typed canonical source states produce the corresponding admitted selected packets without projecting away the original source admission obligations.

At this target calculus's dependent typing layer, an actually typed two-argument data checker determines the native family

$$\Sigma p : \mathsf{Data}.\ \mathsf{Id}_{\mathsf{Data}}(\mathsf{checker}\ j\ p, j).$$

The packing program $\lambda p.\lambda e.(p, e)$ has actual formation-sensitive Pi/Sigma/Id typing. Its application is typed in this family and reduces by two original beta steps to the supplied receipt and equality proof. Renaming and simultaneous substitution commute with the family. This generic construction supplies neither the checker's source authority nor an equality proof for an invalid receipt; those are further obligations of its consumer.

Literal named-constructor images have a stronger conservativity result. Injective constructor names make the source encoding injective, and all such images are normal when the declared root computation is empty. Church–Rosser then proves

$$\mathsf{Conv}_{\mathrm{native}}(\mathsf{encode}(t), \mathsf{encode}(u)) \quad \Longleftrightarrow \quad t = u.$$

Intermediate native terms may lie outside the source image. Finite structural conversion certificates have the same exact endpoint characterization. Controls include a beta-expansion detour outside the image, colliding names that destroy injection, and an added constructor equation that destroys this literal-conversion conclusion. Derived closed operator interpretations are separate: they may intentionally identify source syntax.

The full authored type-expression interpreter covers base sorts, function types, multiple-binder types and collections. All six original binding-syntax operator families have formed native implementations, preserving simultaneous substitution. Collection vectors, bags and sets may share an operational list representation; raw representation equality therefore does not reflect their source types. A checked native packet retains the complete source type index and collection kind, rejecting both outer-kind and nested-type confusion. This packet's sequence, permutation and membership observations are distinct from the source language's separately licensed algebraic equation generators.

For the finite GSLT command catalog, the optional declaration installation gives encoded source values actual target formation judgments and reflects the selected service results to source steps. This result concerns that installation and image. Compiling to a target proof family and proving its ambient conversion properties are separate target-comparison obligations, not missing parts of the independent source judgment's definition.

The linked authored-receipt adapter retains the literal requested claim, complete source header and original receipt. Its success is equivalent to the independent source validity judgment. Its selected empty or actual Atom service handlers must retain their own source admission; built-in Atom carrier admission and an authored plan's structural sort admission are different. This adapter returns target data. It must remain downstream of the original checker, even when dependent packing is studied as an additional interpretation.

Declaration names use reversible structural components, with UTF-8 text and binary numeric fields. Their serialized storage is bounded above and below linearly in the corresponding component-and-payload measure. This avoids repeated numeric-pairing expansion; it is a storage result rather than an executed decoder-time theorem.

Weakness and efficiency as different objectives. Bennett's weakness criterion concerns the generalization of hypotheses under an explicitly specified task model; it is not a theorem identifying a smallest proof kernel [23]. Goertzel's efficient pragmatic intelligence explicitly accounts for computational resources relative to environments and goals [24]. These motivate separate questions here: which assumptions suffice, which inference operations suffice, and what discovery, compilation and checking cost on the intended workload. A proof of one optimum does not identify the others.

10 Resource-bounded reasoning as a consumer

Goertzel's treatment of pragmatic intelligence explicitly accounts for resource use and discusses multiple objectives [22]. The connection here is a proposed engineering use of the preceding theory: a reasoning agent chooses among certified presentations, proof producers, and backends while keeping the logical contract fixed.

For a workload w, consider the vector

$$(\mathsf{searchTime}, \mathsf{translateTime}, \mathsf{checkTime}, \mathsf{bytes}, \mathsf{peakSpace}, \mathsf{authorityAssumptions}).$$

An admissible choice supplies the required preservation, reflection, and observation evidence. Pareto comparison can then discard choices dominated on that workload; a declared policy chooses among the remainder. Actual cost observations and proved bounds play different roles. This defines a possible decision discipline, not a theorem of general intelligence or a

universally optimal proof strategy. Discovery costs can dominate even when certificates and checking are small. The implemented finite-route frontier is deliberately narrower. It starts with one source packet, an already discovered certificate, and a finite family of exact executed compilation/checking routes. Its coordinates are complete target packet symbols, proved translation and checking clocks, and the joined execution's actual peak cells. Every nonempty feasible family has a nondominated route; the selected route retains its source authority and satisfies the composed clock and peak bounds. Increasing the budget retains feasible routes, although it may change the frontier. A checked control has an accepted source theorem but no feasible route under a zero translation budget. Another has two incomparable resource vectors and no least vector. Neither resource infeasibility nor the lack of a universally cheapest route is a logical refutation. These results do not bound the cost of obtaining the profiles or implementing a scheduler. For a declared workload distribution μ and randomized policy A, an additional observable is $F_{A,\mu}(B) = \Pr[\text{a valid certificate is returned within budget } B]$. Budget exhaustion is recorded as pending. Averaging only successful runs hides those cases and can reward a policy that solves few problems. These workload-relative success curves complement worst-case bounds; they do not replace semantic correctness or imply a universal ranking. Authority assumptions are compared by a declared admissibility relation, not assigned a universal numerical trust score. Selecting a fast backend must not silently strengthen the axioms or weaken the required observer.

One useful contract-based optimization is to reuse a checked result at the same provider revision. Another is to specialize a proved pure decision operation. Reusing evidence after a changed snapshot, or treating an external answer as a proved theorem without its stated authority, violates the contract regardless of the apparent speed gain.

11 Existing anchors and the remaining theory

The following names locate existing mathematics in the library. They are not assertions that the full programme below is already formalized.

Existing area	Mathematical role and boundary
`GSLT.LanguageDef.ProofSystems`	Certificate simulations and size bounds; `PSimulation` has a declared, independent time account.
`GSLT.LanguageDef.` `ProofSystemExecution`	`OperationalPSimulation` attaches a physical machine execution.
`GSLT.LanguageDef.` `ResourceTranslation`, `ResourcePreorder`	Uniform whole-packet operational translations, exact/reflection policies, actual identity/copy composition and mutual-simulation equivalence.
`GSLT.LanguageDef.` `ResourceExecution`	Translation/checking and partial-producer/checking in one machine, with physical transfers; polynomial checking transport requires explicit target bounds.
`Computability.ExecutionSpace`, `StackMachineSpaceGrowth`, `StackMachineMicroSpace`	Actual transition and intra-command peaks; a syntactic push bound connects time to output length and all stack cells.
`Computability.` `StackMachineCompositionSpace`	Exact maximum of component transition-state peaks for the original sequential machine, including the constant-cell physical transfer.
`GSLT.LanguageDef.` `ResourceMachineBounds`	Derives operational output-size obligations and polynomial peak space from the implemented clock.
`GSLT.LanguageDef.` `ResourceComputation`	Arbitrary proved executed clocks, their physical composition, and the polynomial subclass.

`GSLT.LanguageDef.ResourceSelection`	Finite feasible-route frontiers retain original authority, executed clock bounds and actual peak storage; no universal optimum or free scheduling algorithm is implied.
`Logic.ProofSearch.CertificateBounds`	Computable exhaustive bounded certificate search; undecidable theoremhood has no computable positive certificate-length bound.
`Logic.ProofSearch.SearchBounds`, `SearchTransport`	Partial-producer budget obstruction; actual forward search/compilation runs and conditional reverse transport requiring an implemented all-proof extractor.
`Logic.ProofSearch.ExecutedBudgets`	Exact completed/live observations of original machine runs, with discovery, compilation, checking and both physical transfers in one executed budget.
`Logic.ProofSearch.FiniteCertificateExecution`	An original-order finite binary enumerator and complete checker loop, with charged candidate storage, refusal work and physical handoff; returns bounded existence.
`Logic.ProofSearch.FirstCertificateExecution`	Retains checked candidates and returns the original first accepted certificate, with exact finite-budget observations and an independent result readout.
`Logic.ProofSearch.ShortCircuitCertificateExecution`, `ShortCircuitBoundedSearch`	Actual first-success checking with no later checker calls, charged suffix cleanup, and a joined bounded enumerator with independent accepted-certificate readout.
`GSLT.LanguageDef.TierLattice`	Elaboration composition, work accounts, and reachability under step coarsening.
`Languages.MM0.Translation`	First-order representation and certificate translations; preserve the hypotheses of each theorem.
`Languages.HOLLight.InferenceCompiler`	`derives_iff_compiled_graph`, independent reflection, and a sharing-preserving compiler for admitted first-order schemas.
`Languages.HOLLight.InferenceCompilerSize`	`derives_iff_serialized` and physical certificate-size bounds; whole-translator executed time is a separate join.
`Languages.HOLLight.AdmittedArgumentRendering`	One finite-alphabet machine admits scope and canonical metadata, renders the original typed HOL argument or refuses, with a common executed cubic bound.
`Computability.FramedWordMapping`, `Languages.HOLLight.ArgumentVectorRendering`	An executed caller retains order, duplicate occurrences and empty refusal fields across complete vectors, with a cubic bound on the framed input.
`Languages.HOLLight.ArgumentListRendering`	The original serialized list is arity-checked and rendered by one finite machine, with a cubic clock including arity and individual-field refusal paths.
`Languages.HOLLight.RuleArgumentRendering`	Original rule lookup, authored arity, complete scope admission and rendering share a cubic clock; retaining the complete source instance also charges its copying. The authored calculus is fixed in this bound. Its executable table is qualified in `RuleArgumentProgram`; `RuleArgumentRetentionProgram` exposes the original source-preserving caller.
`Languages.HOLLight.SourceQuantifierEmission`	Source scope and metadata admission, raw rendering, original naming and the complete eighteen-node SPEC block share one cubic execution bound, including source refusal. The predicate and incoming theorem are fixed parameters.

`Languages.HOLLight.ClaimCheckingMachine`	One finite machine admits source scope, renders the source judgment, projects the independently submitted theorem's actual sequent and compares both hypotheses and conclusion. Its cubic input-byte clock counts the entire theorem, including axiom tags. Claim agreement establishes no derivability or axiom authority by itself.
`Languages.HOLLight.CachedClaimChecking`	Both physical cache tables, source-root resolution, theorem lookup, source admission and the original claim guard compose under one cubic input-byte clock. Missing roots, missing target rows, changed claims and invalid scope produce completed refusal. Stored theorem validity, arbitrary malformed packet admission and the complete chronological driver remain separate obligations.
`Languages.HOLLight.SourceRuleSpecialization`	Original source arguments execute authored arity and scope admission, rendering and naming. The original row and specialization stage determine the changing predicate and incoming theorem; the complete eighteen-node SPEC block has a joined cubic clock with the compiled descriptor charged.
`Languages.HOLLight.DynamicModusPonensEmission`	Original theorem fields execute provenance membership and union over a compact authored basis, then emit the original fifty-eight-node MP block. Upstream operand coherence and an accepted chronological prefix remain explicit obligations.
`Languages.HOLLight.AuthoredNodeDispatch`	The actual source row determines the premise count. One quadratic execution admits the original instance, compares the requested goal, computes its premises and pairs them with the ordered submitted children. It prepares jobs; it does not yet validate those children.
`Languages.HOLLight.GraphAddressAddition`, `Languages.HOLLight.GraphAddressSequence`	Original natural-codec fields execute binary carry and ordered runtime-base offset sequences. Addition takes at most $57N + 64$ steps; sequence cost charges its compiled descriptor. Parent-field rewriting and cache assembly remain separate.
`Languages.MM0.MeTTa.ExecutedSoundness`	Original binary assertion and whole-file theorem acceptance imply original kernel derivability using the exact preceding declaration tables.
`Languages.MM0.MeTTa.MMBProofRunExecution`	Actual quoted reference, save and hypothesis command runs, exact refusal suffixes, and the emitted hypothesis's original kernel validation.
`Languages.MM0.MeTTa.MMBTheoremExecution`	Actual quoted theorem resolution, original kernel validation of typed applications, ordered premise witnesses, chronological lookup and inert proof recording.
`Languages.MM0.MeTTa.MMBTheoremCommandExecution`	Original typed stack consumption, ordered premise projections and saved-proof chronology; storing a witness does not establish its validity.
`Languages.MM0.MeTTa.MMBSlotExecution`, `MMBAssertionMatchExecution`	Exact source slot updates and reference matching by original allocation identifiers.
`Languages.MM0.MeTTa.MMBRecursiveMatchExecution`	Actual recursive constructor/reference matching and ordered hypothesis boundaries, including reserved-slot preservation and total completed observations for original decoded instructions and allocator-shaped expressions. Malformed-argument admission remains separate.

`Languages.MM0.MeTTa.MMBAllocationExecution`	Original variable, dummy and constructor allocations earn complete expression-payload authority. Allocation identifiers remain distinct from physical heap positions, including unsaved expressions and stored witnesses.
`Languages.MM0.MeTTa.MMBMatchAuthority`, `MMBAllocationResolution`	Original reference matching entails complete expression equality when its payloads come from the constructed allocation history. A fixed current declaration resolver connects allocated terms to logical kernel preterms; changed or shadowed resolution can invalidate that authority even when pointer matching still accepts.
`Languages.MM0.MeTTa.MMBAllocationRunExecution`	Arbitrary finite original runs of qualified constructors, dummy allocation and the reference/save/hypothesis fragment preserve chronological payload and resolver authority. Explicit refusal leaves the suffix unread; theorem dispatch and conversion remain further boundaries.
`Languages.MM0.MeTTa.MMBConstructorExecution`	Total original constructor/dummy dispatch under earned allocation/currentness authority, numerical maps and actual list-shaped rows; lookup, declaration-shape and pop refusals preserve the original unread suffix. Raw nonnumeric IDs and non-list rows retain their distinct original behaviours.
`Languages.MM0.MeTTa.MMBTheoremSignatureExecution`	Original ordered dependency ranks, declaration resolution, binder contexts and initial filled slots, inert saved-expression replay and exact unavailable-slot refusal, original fresh-dummy allocation and recursive filled-reference child streams, reserve-before-children term construction and exact self-reference refusal. The independent mutual statement decoder now accepts explicit physical-to-logical term/sort maps, preserves ordered slots, and applies the original dummy guard before each hypothesis. Original context execution now preserves and reflects canonical relocated contexts; bound rank metadata is earned from the original argument initializer. Physical row arity remains distinct from resolved logical IDs. Positive execution of both mutually recursive Polish expression decoders is now proved, retaining the original unread suffix, reservation order and dummy guard. General decoder refusal and the complete theorem-signature wrapper remain open.
`Languages.MM0.MeTTa.MMBRecordedWitnessAuthority`	Prepared original proof references and recorded witnesses retain their earned preterm, declaration and chronological authority. Future references and changed saved claims are refused. A stored proof tag alone supplies no authority.
`Languages.MM0.MeTTa.MMBTheoremMatchExecution`	The actual theorem matcher checks its conclusion and reversed ordered hypotheses, preserving admitted physical slots and allocation authority. Both recursive phases retain their original completed Boolean refusals. Logical validity still comes from checking the recorded initializer.
`Languages.HOLLight.MM0Meaning`, `DefinitionHistoryModels`	Acceptance-to-model meaning and a constructed model for chronological, polymorphic source constant-definition histories.
`Languages.HOLLight.SourceTypeDefinitionTransport`	Original admitted type-definition model transport and both retained equation truths for a fixed instance, preserving old formed source terms.

`Languages.HOLLight.ModelAssignmentSupport`	Earns finite assignment support for the constructed initial model and every admitted polymorphic constant history; arbitrary supplied meanings need not have that support.
`Languages.HOLLight.ModelInstanceCoherence`	Constructs source-domain meanings coherent with original type instantiation, earns defining-body formation from admission, and preserves the old formed source language when a constant history is replayed over a fresh operator extension.
`Languages.HOLLight.ConstantHistoryInstanceCoherence`	Commutes actual admitted constant histories with source type reducts, preserving the original equality and selection declaration licences.
`Languages.HOLLight.UniformTypeDefinitionFamily`	Constructs witnessed predicate subtypes for every original parameter tuple, proves uniform abstraction/representation laws and preserves old source meanings after installation in the operator table.
`Languages.HOLLight.UniformTypeDefinitionModels`	Interprets both new source symbols in one model and proves both original retained equations at every formed public instance, including nested new-type arguments.
`Languages.HOLLight.UniformTypeDefinitionSupport`	Earns assignment support for every installed symbol, compositional term and subsequent admitted constant history from the constructed source model.
`Languages.HOLLight.ConstantHistoryExpandedTypes`	Preserves original constant histories at newly formed carrier types after a type-table extension, including source type-reduct coherence and malformed-instance refusal controls.
`Languages.HOLLight.UniformTypeDefinitionInstances`	Earns licensed source type-instance coherence for the installed uniform model, later type tables and arbitrary following constant histories.
`Languages.HOLLight.UniformTypeDefinitionExtensions`	Preserves the actual witnessed subtype, inhabitant and installed maps when a later fresh type operator is added, including original constant suffixes.
`Languages.HOLLight.MixedDefinitionHistory`	Original mixed admission chronology, formation and translation of all retained equation instances, preservation through constant extensions and the first uniform type model.
`Languages.HOLLight.MixedDefinitionModels`	Constructed full-domain models for arbitrary original admitted constant/type histories, uniform retained equations, initial truth preservation and relative consistency. The broader universal-predecessor chronology is constructed separately.
`Languages.HOLLight.DefinitionHistoryMeaning`	Configured saved MM0 acceptance entails truth in the constructed model of the existing mixed history, now using all retained universally closed schemas. Its type-existence admission retains that history's original restriction.
`Languages.HOLLight.UniversalDefinitionHistory`	A broader original definition chronology admits type-existence proofs from the actual preceding universal background. Restricted histories embed into it, and source substitution preserves the background.
`Languages.HOLLight.UniversalDefinitionModels`	Constructs the model through the broader history, interpreting each preceding existence proof to earn the subtype witness and preserving all retained schema instances.

`Languages.HOLLight.UniversalDefinitionMeaning`	Original configured MM0 acceptance implies truth in that constructed model under the translated hypotheses. Actual validated type-and-term substitutions preserve acceptance; every certificate for falsity is refused. Its authorized axiom rows are precisely the declaration rows; adding the standard HOL axiom rows is a separate persistent-truth and policy join.
`Languages.Megalodon.NormalizationGrowth`	Original typed normalization, exact full-codec growth, accepted conversion chains, and an output-based obstruction to polynomial expanded normalization.
`Languages.Megalodon.NormalizationSharing`	Original graph materialization, polynomial complete shared storage, and a lower bound on emitting its expanded tree serialization.
`Logic.HOL`, `Languages.OpenTheory`	Model interpretations, soundness, and operational primitives; no automatic assertion of completeness of the original HOL Light kernel for all Henkin models.
`Logic.HostingStyles`, `Logic.Consequence`	Typed theories, interpretations, and proof fibres to reuse for semantic composition.
`OSLF.NativeTypes`, driver bridges	Independently defined observations and evidence transport on their admitted profiles.

The resource interfaces above extend the original authority objects rather than changing their semantics. Configurations, claims and certificates occur in the encoded packet. Exact replay includes rejected certificates in the compiler image, while arbitrary-target reflection is a separate property. A checked counterexample has exact image replay but an additional host rule, so it lacks global reflection and an all-proof extractor. A physical control bounds both acceptance and refusal by nine steps: four for copying, four for handoff, and one for the checker. The internal stack alphabets are required to be finite in the operational authority interface.

The argument-vector join is a concrete instance of executed composition. For fields with encoded lengths n_i, emitted lengths m_i and original scalar clock bounds c_i, its clock is

$$2 + \sum_i (c_i + 2n_i + 2m_i + 4).$$

The caller physically transfers each input, collects each output and reverses the complete accumulator. A refused scalar argument contributes an empty framed field at its original index, distinct from an absent field. The instantiated HOL renderer has a proved cubic upper bound in the complete framed input length. Independent decoding of any completed run recovers the original per-field accepted term or refusal. The decoder also distinguishes malformed framing from a correctly framed refused field.

The list-level join begins with the original serialized Pattern list. The existing list framer earns the declared arity before dispatching to the vector renderer. A physically emitted final status distinguishes a wrong arity from a successful empty vector, while inner refusal fields retain their positions. For a fixed authored arity and original input length N, the joined machine has the executed bound

$$\mathsf{steps} \leq 3000200000(N+1)^3.$$

The bound charges both intermediate transfers, scope admission, rendering, status emission and cleanup. Its independent output decoder distinguishes malformed framing, arity refusal and individual-field refusal. This is still a qualified argument subroutine; generated axiom fields and the complete certificate driver require their own joins.

The authored-rule join additionally starts from the original complete rule instance. It executes first-match lookup, authored arity, every argument's scope admission and rendering in one clock. For each fixed authored calculus its bound is $3000300000(N+1)^3$; retaining the complete original instance raises this to $3000500000(N+1)^3$ and charges the extra physical copying. Missing rules, duplicate-name lookup, wrong arity and invalid unused arguments are covered by the same execution law. Rule side conditions and child judgments remain obligations of the full certificate driver.

The executable table is qualified separately from the mathematical finite machine record. Structural instruction embeddings remove enumeration data for finite control and register types from runtime arguments; checked transition and initialization equalities preserve the original machine's steps. This representation change does not turn the abstract step bound into a wall-clock or native-allocation bound, and compiling the authored rule table remains a separate uniformity obligation.

The SPEC field serializer now constructs all eighteen original primitive occurrences for a fixed typed predicate and incoming theorem. Its three independent dynamic fields contain the complete raw operand, its original named representative and the occurrence origin. The output agrees byte for byte with the existing checked chronological expansion, including its intermediate claims, parent references, hypotheses and axiom dependencies. For fixed template length L and supplied field-packet length N, its actual indexed machine runs in at most $4(L+1)(N+1)$ steps. The independent certificate decoder recovers the checked block. This field serializer alone assumes that the supplied named representative matches the raw operand. A higher-order operand control produces 140,148 bytes in 318,520 counted transitions and replays through the original checker. Future roots, a self-parent, inconsistent raw/named fields and trailing bytes are refused.

The new naming machine derives the original named operand from its raw applicative spine in at most $3n + 4$ transitions. Constant names, type annotations and provenance payloads are copied unchanged, including lambda syntax stored inside opaque source data. Its domain includes every rendered source argument. Five positive and six refusal execution controls cover nested applications, repeated ordered children, Unicode, opaque binders and definition provenance; arbitrary higher-order term spines are outside this particular naming pass.

A single composed machine now starts with only the raw rendered argument and origin. It derives the named field, retains and frames the two supplied fields, and emits the original SPEC block. If L is the fixed permuted emission-template length and N the two-field input length, the proved clock is $(8(L+1)+64)(N+1)$, including both computations and every physical handoff. Every terminating run on this input domain decodes to an original checked eighteen-node block. This stage's input remains the raw operand; the next composition adds source admission and rendering. Two executions start with an opaque source binder and repeated ordered arguments respectively. They take 424,819 and 849,587 transitions, emit 154,940 and 209,792 bytes, and replay as the original eighteen-node blocks with clean auxiliary stacks. Eight mutations of references, trailing bytes and argument fields are refused by the independent readout and checker.

The source-input composition now starts with the complete original Pattern and origin. It checks scope and canonical binder metadata, renders and parses the canonical HOL operand, computes its named form and emits SPEC, with every physical handoff included. Its clock is

$$\big(3001030000 + 60000(L+1)\big)(N+1)^3,$$

where N measures the source packet and L the fixed permuted emission template. Rejected scope or metadata produces no decodable certificate. Every accepted terminating readout earns source admission and the original relative checker judgment, including all eighteen origins. Three compiled readout controls pass original replay, four invalid source arguments

refuse, and six certificate mutations refuse. These thirteen controls exercise the semantic readout; the general clock is established by the kernel-checked execution theorem, rather than a measured physical run of this larger composition. Arbitrary malformed-wire admission, changing-predicate control generation and the whole translator driver remain further obligations.

The changing-predicate composition is now established for an original authored row and a compiled specialization stage. Runtime input consists only of the original Pattern argument list and occurrence origin. The machine executes the original argument count and scope checks, renders and names each operand, retains the complete ordered fields, selects the proper prefix and current argument, and emits all eighteen original SPEC nodes. The incoming theorem and predicate are generated from the row and earlier arguments. An invalid argument that the rule body does not use still refuses. Its cubic bound includes the actual compiled descriptor; generating that descriptor from a runtime calculus remains a separate obligation.

Authored node dispatch now executes original rule-name selection and uses that row's premise count. Its child preparation computes the original instantiated premises, compares the complete requested conclusion, and pairs the premises with submitted proofs in order. The quadratic clock includes argument admission, repeated instantiation, copying and refusal cleanup. An independently decoded successful output earns an original source rule application and exact child occurrences. Child-proof validity, graph insertion and chronological cache invariants are not inferred from that preparation result.

Graph-address addition now reads two original natural-codec fields and physically propagates binary carries before restoring the original output codec. The $57N + 64$ bound depends on encoded lengths, including numbers beyond fixed machine integers. Twelve complete instruction runs check original decoding, exact original transition counts and scratch cleanup. A fixed offset vector of descriptor length D has an actual composed sequence machine, with bound $80(D+1)(N+1)$ on a runtime graph base of encoded length N. Its independent readout preserves order and duplicates, and its internal-parent addresses agree with the original block relocation. Eleven complete sequence instruction runs check exact ordered readouts, including eighteen- and fifty-eight-node offsets and 257-bit addresses. External-root lookup authorization, whole block-relocation execution qualification and chronological insertion remain further joins.

For a compiled block topology with admitted parent and root bounds, physical relocation now selects the original runtime base, earns all internal addresses, retains the original payload and external-root fields, and emits the complete original certificate codec. The resulting word decodes to the original block expansion. Original replay supplies the topology bounds; checked earlier-root lookup and a checked local block supply original checker acceptance after insertion. The step bound is $128(D+1)(L+1)(N+1)$, charging both the address-plan length D and relocated-emission template length L. This composition has a checked execution proof and polynomial realization. Whole instruction-run qualification, earlier-root authorization, malformed packet admission and chronological cache insertion remain separate joins.

These bounds concern encoded packets. They do not by themselves qualify malformed byte admission, physical bit costs, native allocator space, or an unimplemented source-system instance. The whole-file MM0 theorem retains its declaration-correspondence hypotheses; the HOL model theorem retains substitution-closed background, configured axiom-schema and source-hypothesis obligations. A configured false axiom and model-valid but bare-kernel-unprovable eta judgment supply negative controls for stronger semantic claims. The constant-history model is built from the actual source admission receipts, preserving old terms and proving every recorded defining equation at every type instance. Reused names and references to a future definition are refused. The separate type-definition transport constructs the admitted subtype carrier for a fixed instance, installs the original abstraction and representation symbols, preserves old formed terms, and proves both actual retained defining equations. Re-

declaration is refused. The assignment-support development additionally proves that changes outside a source symbol's public type parameters preserve its constructed meaning throughout an admitted constant history. The original no-hidden-parameter admission guard earns this property at each extension; arbitrary supplied meanings have a checked counterexample. Assignment locality alone does not imply instantiation coherence: the earlier intrinsic default interprets a Boolean proposition as true, while a type parameter assigned the original Boolean domain retains its false inhabitant. A checked counterexample exhibits the mismatch. The new coherent interpretation instead uses each complete evaluated source domain's inhabitant. Actual declaration licences then earn source-instantiation agreement. Original constant admission also earns formation of every used replacement in its defining body, without requiring unused substitution entries to be formed. Replaying an entire constant history over a fresh operator table preserves all old formed source meanings. Constant histories now also commute with source type reducts, including dependency-bearing polymorphic definitions. The proof retains actual equality and optional selection declaration licences; unlicensed annotations have a checked negative example. The uniform type-family constructor now reconstructs the preceding constant history at each admitted domain tuple. An actual syntactic proof of the retained existence application earns nonemptiness, and the original parameter guards earn assignment locality. Both constructed abstraction/representation laws hold for every admitted tuple; wrong arities are refused. Installing the family into the full operator table preserves the old formed source language. One uniform model now interprets the original new abstraction and representation symbols and validates both original retained equations at every formed public instance. Equal public annotations force equal complete domain tuples, earning independence of the substitution witness used to interpret the symbols. Replacement arguments may contain the new operator itself. Boolean, function and nested new-type controls share one model; malformed operator arity and unlicensed public annotations are refused. Assignment support for all installed symbols, compositional terms and arbitrary following admitted constant histories is now earned from the complete parameter tuples. Changing a used domain inhabitant has a checked negative control. Licensed source type-instantiation coherence is now proved for the installed model, even when the public registry subsequently admits more types, and through arbitrary following original constant histories. The proof recovers each original declaration licence and commutes its actual abstraction/representation value with composed type substitutions. An unlicensed raw annotation can become recognized after substitution, so the licence cannot be dropped. A later fresh operator now preserves the actual witnessed subtype, its selected inhabitant and the installed abstraction/representation maps, including original constant suffixes. Admission earns omission of the fresh operator from the defining predicate and represented type. Arbitrary repeated mixed type/constant histories now retain their exact admission chronology: each existence proof uses only the preceding original equations. All retained equations remain formed and translate at every type substitution; later constant extensions preserve them. The first uniform type model satisfies all preceding constant equations and both new type equations, including future replacement annotations. A dependency-bearing type–constant–type sequence and premature/repeated admission refusals are checked. The semantic recurrence is now constructed over arbitrary original admitted histories. Every type step builds a genuine witnessed subtype family, installs the exact original abstraction and representation Symbols at all licensed annotations, and earns support, later-registry instance coherence and fresh-operator preservation again. A constant after two type extensions validates all eight retained rows. No final model or coherence premise is supplied to the recurrence.

This construction proves all retained equations, soundness of their exact equation theory, preservation of formed initial-language truth and relative consistency, with explicit initial primitive licences and ambient families. The acceptance-to-truth interface requires a variable-free, source-substitution-closed background. Pointwise retained type equations contain free

source variables, so they do not directly supply this interface. The original two type equations now have explicit universally closed schemas. All-elimination derives each pointwise equation; original admission earns absence of free source variables, and the constructed model validates both schemas for every bound value. Source-agreement transport now supplies the matching bound valuations after every source type substitution in an arbitrary mixed history. It earns both universally closed instantiated schemas without assuming that unused replacements are formed. The actual target substitution, including symbol replacement, now agrees with type substitution on admitted closed ordinary schemas. Their iterated type images give a genuine variable-free substitution-closed background, all true in the constructed installed model; universal elimination proves the original two returned rows. The broader original chronology now uses this preceding universal background for each existence proof. A history-independent model family is extended through every actual admission: the preceding syntactic proof is interpreted to obtain a witness, and the subtype installation preserves every retained schema instance. The resulting full-domain model validates the entire background and all original finite rows. Configured saved-MM0 acceptance therefore entails truth under the translated hypotheses, including after actual validated `INST_TYPE`/`INST` steps. This construction assumes neither a final model nor truth of the existence witnesses. The configured policy here authorizes precisely the declaration rows. Incorporating the standard HOL axioms requires their actual policy membership and persistent truth through the history, including the appropriate initial infinity witness. The genuine export chronology and complete operational translator also remain external joins.

The theory should extend these abstractions rather than replace them. The name "native inference kernel" records a programme of parametric native checking; it is neither a synonym for replay correctness nor a theorem of universality. These notes do not resolve or redefine that programme. Its historical account belongs with the existing library documentation.

Remaining theorem packages. (1) Complete real-proof end-to-end qualification on the original checkers, including shared claims and refusal paths. (2) Assemble executed translation bounds for every arrow used in an operational polynomial-equivalence claim, with physical handoffs. (3) Extend hosting to binding rule schemas with typed adequacy and explicit substitution evidence. (4) Join signature/module interpretations and model transport, including conservative definition histories. (5) Complete source-visible whole-run GSLT observations and their native type transport, including effects and provider revision. (6) Prove the optimization contracts and measure whole costs.

Existing capacity theorems support this programme, but do not discharge its remaining packages. A real HOL proof checked independently is a valuable integration control; it does not replace an arbitrary-input correctness theorem. Conversely, a polynomial certificate theorem does not make a large emitted specification convenient. Verified extraction to a costed intermediate language is a promising way to reduce repeated hand-built machine proofs [16]; its use requires an actual verified connection to the current translator and chosen machine model. The time-and-space invariance result for weak call-by-value lambda calculus [17] is another useful input, but its displayed Boolean-output theorem does not by itself supply an output-sensitive bound for writing arbitrary translated proof certificates.

References

[1] S. Cook and R. Reckhow. The relative efficiency of propositional proof systems. JSL 44(1), 1979. doi:10.2307/2273702.

[2] S. Buss. On Gödel's theorems on lengths of proofs I: Number of lines and speedup for arithmetics. JSL 59(3), 1994. doi:10.2307/2275906.

[3] G. Burel. Efficiently simulating higher-order arithmetic by a first-order theory modulo. LMCS 7(1), 2011. arXiv:0805.1464.

[4] M. Carneiro. Metamath Zero: Designing a theorem prover prover. CICM 2020. Preprint titled *The Cartesian Theorem Prover*: arXiv:1910.10703.

[5] J. Goguen and R. Burstall. Institutions: Abstract model theory for specification and programming. JACM 39(1), 1992. doi:10.1145/147508.147524.

[6] L. Henkin. Completeness in the theory of types. JSL 15(2), 1950. doi:10.2307/2266967.

[7] J. Väänänen. Second-order logic and foundations of mathematics. BSL 7(4), 2001, 504–520. https://www.mv.helsinki.fi/home/jvaanane/secondorder.pdf.

[8] G. Dowek, T. Hardin, and C. Kirchner. HOL-$\lambda\sigma$: An intentional first-order expression of higher-order logic. MSCS 11(1), 2001. doi:10.1017/S0960129500003236. Accessible precursor: INRIA RR-3556, November 1998, https://inria.hal.science/inria-00077202/file/RR-3556.pdf.

[9] R. Harper, F. Honsell, and G. Plotkin. A framework for defining logics. JACM 40(1), 1993. doi:10.1145/138027.138060.

[10] M. Fiore, G. Plotkin, and D. Turi. Abstract syntax and variable binding. LICS 1999. doi:10.1109/LICS.1999.782615.

[11] M. Fiore and D. Szamozvancev. Formal metatheory of second-order abstract syntax. POPL 2022. arXiv:2201.03504.

[12] R. Arthan. On definitions of constants and types in HOL. JAR 56, 2016, 205–219. doi:10.1007/s10817-016-9366-4.

[13] R. Kumar, R. Arthan, M. Myreen, and S. Owens. Self-formalisation of higher-order logic: Semantics, soundness, and a verified implementation. JAR 56, 2016. doi:10.1007/s10817-015-9357-x.

[14] R. Statman. The typed λ-calculus is not elementary recursive. TCS 9, 1979. doi:10.1016/0304-3975(79)90007-0.

[15] B. Accattoli and U. Dal Lago. (Leftmost-outermost) beta reduction is invariant, indeed. LMCS 12(1), 2016. doi:10.2168/LMCS-12(1:4)2016.

[16] Y. Forster and F. Kunze. A certifying extraction with time bounds from Coq to call-by-value lambda calculus. ITP 2019. doi:10.4230/LIPIcs.ITP.2019.17.

[17] Y. Forster, F. Kunze, and M. Roth. The weak call-by-value lambda-calculus is reasonable for both time and space. POPL 2020. doi:10.1145/3371095.

[18] The Lean Language Reference. The type system. https://lean-lang.org/doc/reference/latest/The-Type-System/.

[19] M. Carneiro. Lean4Lean: Verifying a typechecker for Lean, in Lean. 2024; revised 2025. arXiv:2403.14064.

[20] A. Abel and T. Coquand. Failure of normalization in impredicative type theory with proof-irrelevant propositional equality. LMCS 16(2), 2020. arXiv:1911.08174.

[21] T. Coquand. Reduction free normalisation for a proof irrelevant type of propositions. LMCS 19(3), 2023. arXiv:2103.04287.

[22] B. Goertzel. The general theory of general intelligence: A pragmatic patternist perspective. 2021; revised 2026. arXiv:2103.15100.

[23] M. T. Bennett. The optimal choice of hypothesis is the weakest, not the shortest. 2023. arXiv:2301.12987.

[24] B. Goertzel. Toward a formal characterization of real-world general intelligence. AGI 2010, 74–79. doi:10.2991/agi.2010.17.

[25] Megalodon source, revision `96d559ea102496ab50a4102d59032fd6e0845d32`. `src/mathdata.ml`, conversion and resource checks.

[26] Megalodon source, same revision. `src/mathdatapfg.ml`, normalization pass accounting.

[27] Lean 4.33.1 source. `Lean/CoreM.lean` and `kernel/type_checker.cpp`.

[28] E. Frittaion. Completeness of the primitive recursive ω-rule. Archive for Mathematical Logic 59, 715–731, 2020. `https://doi.org/10.1007/s00153-020-00716-9`.

[29] M. Hutter. The fastest and shortest algorithm for all well-defined problems. IJFCS 13(3), 2002, 431–443. Author report IDSIA-16-00, 3 March 2001. arXiv:cs/0206022; `https://www.hutter1.net/ai/pfastprg.pdf`.